ANDROID PRACTICE PROJECT — COMPLETE BUG REPORT

CryptoApp: Full Crash Post-Mortem & Architecture Documentation

Three consecutive runtime crashes diagnosed, explained in depth, and resolved. Covers Hilt DI, Gradle Plugin configuration, AGP 9 compatibility, and Kotlin Serialization.

Crash #1 Missing @AndroidEntryPoint — IllegalStateException

Exception:

```
java.lang.IllegalStateException: Given component holder class com.example.cryptoapp.MainActivity does not implement interface dagger.hilt.internal.GeneratedComponent or interface dagger.hilt.internal.GeneratedComponentManager
```

at dagger.hilt.EntryPoints.get(EntryPoints.java:62)
at androidx.hilt.lifecycle.viewmodel.compose.HiltViewModelKt.rememberHiltViewModelFactory(HiltViewModel.kt:99)

Root Cause

When `hiltViewModel()` is called inside a Composable, it locates the host `Activity` via `LocalContext.current` and casts it to `GeneratedComponentManager` to retrieve the Hilt-managed `ViewModelProvider.Factory`. `MainActivity` was missing the `@AndroidEntryPoint` annotation — meaning Hilt's annotation processor never generated the `Hilt_MainActivity.java` base class that implements this interface.

Fix Applied

```
// MainActivity.kt
@AndroidEntryPoint  // ← Added
class MainActivity : ComponentActivity() { ... }

// CoinApplication.kt — also required
@HiltAndroidApp
class CoinApplication : Application()

// AndroidManifest.xml — register the custom Application
<application android:name=".CoinApplication" ... >
```

Crash #2 Missing Hilt Gradle Plugin & KSP — Still IllegalStateException

> **Exception (same as Crash #1 — same message, different root cause):**
> `java.lang.IllegalStateException: Given component holder class com.example.cryptoapp.MainActivity does not implement interface dagger.hilt.internal.GeneratedComponent`

Root Cause — The "Silent Compiler" Trap

Even after adding `@AndroidEntryPoint` to the source code, Hilt crashed again with the exact same error. The annotation was present, but **no code generation had run**.

> **Critical distinction:**
> `implementation("com.google.dagger:hilt-android: ... ")` only imports the Hilt *runtime library and annotation definitions*. It does **not** perform any code generation.
>
> Two separate build-time tools are mandatory:
>
> 1. **KSP Compiler** (`ksp("com.google.dagger:hilt-compiler: ... ")`) — Generates `Hilt_MainActivity.java` , Dagger component factories, and ViewModel modules.
> 2. **Hilt Gradle Plugin** (`id("com.google.dagger.hilt.android")`) — Runs ASM bytecode transformation (`:transformDebugClassesWithAsm`) to make `MainActivity.class` extend `Hilt_MainActivity` at the bytecode level.
>
> Both were absent from `build.gradle.kts` .

```
@AndroidEntryPoint in source code (annotation present ✓) | ├── WITHOUT Hilt Plugin + KSP: | ├── kspDebugKotlin: SKIPPED → No Hilt_MainActivity.java generated ❌ | ├── hiltCollectClassesDebug: SKIPPED ❌ | ├── transformDebugClassesWithAsm: SKIPPED ❌ | └── Runtime: MainActivity is plain ComponentActivity → CRASH 💥 | └── WITH Hilt Plugin + KSP: ├── kspDebugKotlin: RUN → Hilt_MainActivity.java generated ✅ ├── hiltCollectClassesDebug: RUN → Dagger graph assembled ✅ ├── transformDebugClassesWithAsm: RUN → Bytecode injected ✅ └── Runtime: MainActivity implements GeneratedComponentManager → SUCCESS 🎉
```

An additional complexity arose because **AGP 9.3.3** (used by this project) broke compatibility with Hilt 2.51.1 — it dropped internal extension APIs that Hilt's plugin relied on (`Android BaseExtension not found`). The resolution required upgrading to Hilt 2.60.1 and adding AGP 9 compatibility flags.

Fix Applied

```
// Root build.gradle.kts
plugins {
    id("com.google.dagger.hilt.android") version "2.60.1" apply false
    id("com.google.devtools.ksp") version "2.2.10-2.0.2" apply false
}

// app/build.gradle.kts
plugins {
    id("com.google.dagger.hilt.android")
    id("com.google.devtools.ksp")
}
dependencies {
    implementation("com.google.dagger:hilt-android:2.60.1")
    ksp("com.google.dagger:hilt-compiler:2.60.1")
}

// gradle.properties — AGP 9 compatibility
android.newDsl=false
android.disallowKotlinSourceSets=false
android.sync.suppressAgpWarnings=UNSUPPORTED_PROJECT_OPTION_USE
```

Crash #3 Missing Serialization Compiler Plugin — SerializationException

Exception:

```
kotlinx.serialization.SerializationException: Serializer for class 'CoinListScreen' is not found.
Please ensure that class is marked as '@Serializable' and that the serialization compiler plugin is applied.
 at kotlinx.serialization.internal.Platform_commonKt.serializerNotRegistered(Platform.common.kt:90)
 at androidx.navigation3.runtime.serialization.NavKeySerializer.serialize(NavKeySerializer.android.kt:93)
 at androidx.navigation3.runtime.serialization.NavBackStackSerializer.serialize(NavBackStackSerializer.kt:55)
 at androidx.compose.runtime.saveable.SaveableHolder.onRemembered(RememberSaveable.kt:303)
```

Root Cause

When the app launched successfully (Hilt no longer crashing), AndroidX Navigation 3 attempted to **save and restore the navigation backstack** via the standard Compose `rememberSaveable` mechanism. The backstack is typed as `List<Screen>`, where `Screen` is a `sealed interface` marked with `@Serializable`.

Navigation 3's `NavBackStackSerializer` tried to serialize `CoinListScreen` (the initial route) using `kotlinx.serialization`. The serializer lookup failed at runtime because **no serializer class existed for `CoinListScreen`**.

The "Silent Annotation" Trap (again, but for Serialization):
The `@Serializable` annotation on `Screen`, `CoinListScreen`, and `CoinDetailScreen` was correct in the source code. However, kotlinx.serialization works exactly like Hilt — the annotation is only a marker.

The **Kotlin Serialization Compiler Plugin** (`org.jetbrains.kotlin.plugin.serialization`) must be applied in `build.gradle.kts` for the compiler to generate the actual `serializer()` functions and `$serializer` companion objects for each `@Serializable` class. Without the plugin, `@Serializable` is completely ignored at compile time.

How Navigation 3 Uses Serialization

```
rememberNavBackStack(Screen.CoinListScreen) | └── Compose rememberSaveable saves backstack on configuration change / process death | └── NavBackStackSerializer.serialize(CoinListScreen) | ├── WITHOUT Serialization Plugin: | └── kotlinx.serialization.serializer<CoinListScreen>() → NOT FOUND ❌ CRASH 💥 | └── WITH Serialization Plugin: └── Compiler generated CoinListScreen.$serializer → FOUND ✅ SUCCESS 🎉
```

Fix Applied

```
// gradle/libs.versions.toml — register the plugin alias
[plugins]
kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref = "kotlin" }

// Root build.gradle.kts — declare without applying
plugins {
    alias(libs.plugins.kotlin.serialization) apply false
}

// app/build.gradle.kts — apply the plugin
plugins {
    alias(libs.plugins.kotlin.serialization)  // ← Added
}
```

The `kotlinx-serialization-json` runtime dependency was already present. No changes to `Screen.kt` were needed — the annotations were always correct.

Final Build Verification

After all three fixes were in place, the full Gradle build ran and succeeded:

Task	Purpose	Result
`:app:kspDebugKotlin`	KSP processes Hilt annotations → generates `Hilt_MainActivity.java`, ViewModel modules	**SUCCESS**
`:app:compileDebugKotlin`	Kotlin compiler + serialization plugin generates `$serializer` for `Screen`, `CoinListScreen`, `CoinDetailScreen`	**SUCCESS**
`:app:hiltCollectClassesDebug`	Hilt discovers all `@HiltViewModel` and `@Module` classes	**SUCCESS**
`:app:transformDebugClassesWithAsm`	Hilt's ASM injects `GeneratedComponentManager` into `MainActivity.class` at bytecode level	**SUCCESS**
`:app:assembleDebug`	Full APK compilation and packaging	**BUILD SUCCESSFUL (41 tasks)**

Key Lessons for Future Android Projects

Annotation	Runtime Library	Compiler Plugin Required	Consequence if Missing
`@HiltAndroidApp` `@AndroidEntryPoint` `@HiltViewModel`	`hilt-android`	`com.google.dagger.hilt.android` (Gradle plugin) + `ksp(hilt-compiler)`	No code generation → `GeneratedComponentManager` not found at runtime
`@Serializable`	`kotlinx-serialization-json`	`org.jetbrains.kotlin.plugin.serialization`	No serializer generated → `SerializationException: Serializer not found` at runtime

CryptoApp Practice Project • Android Jetpack Compose + Hilt + Navigation 3 + Kotlin Serialization Updated: November 2026